

Snowflake End-to-End: Real-Time Problems and Agent Playbook

snowflake-ai-data-agent · 95 problems data engineers and analysts hit on this stack, the agent that handles each one, and how to train those agents

What this document is

The stack is Alpaca (trades, bars, news) → Kafka → Snowpipe Streaming and GCS batch loads → Bronze → Streams and Tasks → Silver → dbt → Gold → Semantic View and Cortex Search → Cortex Agent → MCP, Streamlit and FastAPI. Every stage has its own way of failing quietly. This playbook lists those failures as engineers and analysts actually meet them, assigns each one to an agent, and sets out how each agent is trained and tested before it is trusted.

Status: none of these agents exist yet. The repo already has some foundations they would build on: DLQ routing, the `is_suspect` flag, the ONNX anomaly model, the Row Access Policy, the audit-log table and the `sql/08` validation queries. Only the DLQ routing, sample data and ONNX training are verified; everything else has not run against live Snowflake. This is a plan, not a description of working software.

Who feels which problems

- **Data engineers** own freshness, completeness, capacity and cost: feeds that stall, servers that go down, volume spikes that overload every component, timestamps that don't line up, tasks that suspend, tests that fail overnight, grants that disappear.
- **Analysts** own trust in the numbers: ambiguous definitions, confident wrong SQL, delayed data that looks broken, news citations that point to the wrong company, and waiting on engineering for every new metric.
- Many problems belong to both: a data engineer's silent NULL becomes an analyst's wrong average.

What “training” means for these agents

Most of these agents are LLM agents. They get better through what they are given and how they are tested, not by retraining model weights. Training each agent has five parts:

- **Knowledge.** Runbooks, schemas, metric definitions and error catalogues, written down in the repo.
- **Tools.** A small set of narrow, mostly read-only tools (listed per agent below). Anything that changes state is approval-gated.
- **Scenarios.** Faults injected on purpose, using the repo's own sample-data generators and replay script, so every problem in this document becomes a repeatable test case with a known right answer.
- **Evals.** A scored test set per agent, run on every change to its instructions or tools. An agent is promoted only when it passes.
- **Feedback.** Every human correction in production becomes a new eval case, so the same mistake is caught next time.

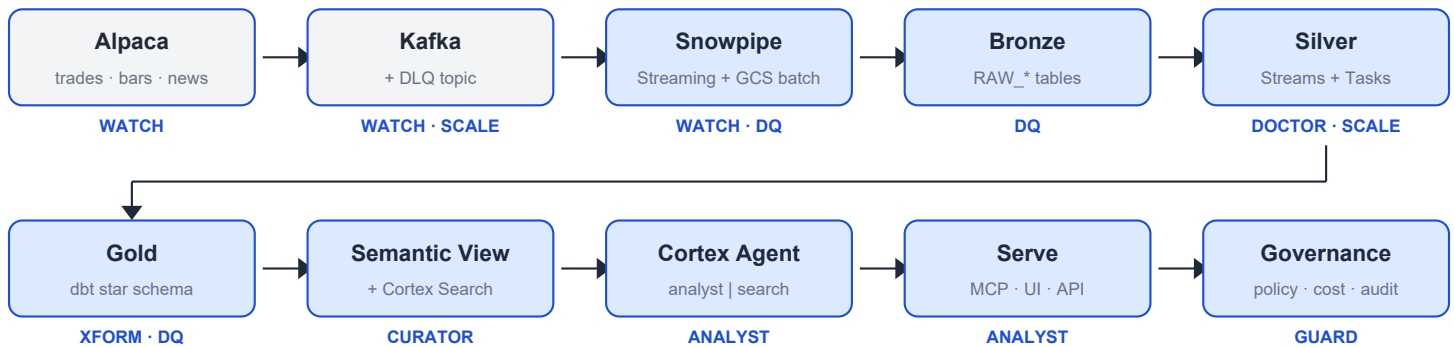
Real model training is used in only two places: the anomaly-detection model (the IsolationForest exported to ONNX, already in snowpark/) and, if evals plateau, fine-tuning for text-to-SQL or routing. Fine-tuning comes last, not first.

Autonomy levels

Level	What the agent may do	Used for
L0 Observe	Read metrics and logs, raise alerts.	Every agent on day one.
L1 Diagnose	Explain root cause with evidence; recommend a fix.	Default level for most problems.
L2 Act with approval	Run a runbook step after a human clicks approve.	Restarts, backfills, resuming tasks.
L3 Act within bounds	Run a pre-approved runbook step automatically, once, then escalate.	Producer restart, alert suppression on closed markets.

Snowflake end to end

The whole pipeline, from the market feed to the analyst's answer. Everything after Kafka runs inside Snowflake, so most problems can be detected, diagnosed and fixed with Snowflake's own objects and views. Problem IDs refer to the tables later in this document.



Grey = outside Snowflake · Blue = inside Snowflake · Labels under boxes = agent watching that stage. TRIAGE and SCALE cover all of them.

Stage	Snowflake objects in this repo	Problems	Agent	Signal the agent reads
1. Source	Outside Snowflake: Alpaca websocket + REST; Kafka topics market-trades, market-bars, market-data-dlq	A1–A9, B1–B5, I1–I4, J1–J3, K2, L1	WATCH, SCALE	Kafka lag + Connect status and Alpaca clock (external); results written into a Snowflake OPS table
2. Ingest	Snowpipe Streaming channels (Kafka connector); GCS storage integration, RAW_STAGE, BARS_BACKFILL_PIPE, SYMBOLS_PIPE, NEWS_PIPE	C1–C5, I5, J4, J6, J7, K7	WATCH, DQ, SCALE	SNOWPIPE_STREAMING_* history, COPY_HISTORY, SYSTEM\$PIPE_STATUS
3. Bronze	BRONZE.RAW_TRADES, RAW_BARS, RAW_SYMBOLS, RAW_NEWS (permissive VARCHAR)	A3, A8, C1, D4, J8, K1, L2	DQ	Data Metric Functions: ROW_COUNT, DUPLICATE_COUNT, FRESHNESS
4. Silver	Streams RAW_*_STREAM + Tasks MERGE *_TASK → SILVER.TRADES, BARS, SYMBOLS, NEWS; SYMBOLS_DT	D1–D5, I6, K5, K6, M1, M2	DOCTOR, SCALE	SHOW STREAMS (stale_after), TASK_HISTORY, DYNAMIC_TABLE_REFRESH_HISTORY
5. Gold	dbt → STAGING views; GOLD.DIM_SYMBOLS, DIM_DATE, FCT_TRADES, FCT_BARS (is_suspect flag; row access policy via post-hook)	E1–E6, A5, A6, A9, K3, K4, K10, L3–L6, M3–M6, J9	DOCTOR, XFORM, DQ	dbt run_results, POLICY_REFERENCES, NULL_COUNT DMFs, ONNX anomaly score
6. Semantic + search	GOLD.MARKET_SEMANTIC_VIEW; semantic_model.yaml; SILVER.NEWS_SEARCH_SERVICE (Cortex Search)	F2–F5, F8, N2, N4	CURATOR	Verified-query regression against sql/08; search refresh lag
7. Agent	AGENTS.MARKET_DATA_AGENT (Cortex Agent with analyst + search tools), role ANALYST_AGENT	F1–F9, I8, J5, K9, N1, N3, N6	ANALYST	GOVERNANCE.AGENT_QUERY_AUDIT_LOG, Cortex usage views
8. Serve	MCP server, Streamlit, FastAPI, all querying as ANALYST_AGENT through the same policy	F6, F9, G2, I7, N5	ANALYST, GUARD, SCALE	Latency and error logs in an event table
9. Govern + cost	Roles, DELAYED_DATA_POLICY, ANALYST_WH, ANALYST_AGENT_MONITOR, audit log	G1–G5, E4, I9, K8	GUARD	GRANTS_TO_ROLES, ACCESS_HISTORY, WAREHOUSE_METERING_HISTORY, resource monitors

The agent roster

Nine agents, each with one job. TRIAGE coordinates the others; ANALYST is the Cortex Agent the repo already defines in `sql/10_agent`.

Agent	Owns	Tools it is given
WATCH Ingestion Watchdog	Alpaca feed, Kafka, Kafka Connect, Snowpipe Streaming, GCS/Snowpipe batch loads	Alpaca /v2/clock + /v2/calendar; Kafka consumer-group lag and topic offsets (read-only admin API); Kafka Connect REST status; SNOWPIPE_STREAMING_CLIENT/CHANNEL history; SYSTEM\$PIPE_STATUS; COPY_HISTORY; DLQ topic reader
DOCTOR Pipeline Doctor	Streams, Tasks, Dynamic Tables, dbt runs	SHOW STREAMS (stale flag); TASK_HISTORY; DYNAMIC_TABLE_REFRESH_HISTORY; dbt run_results.json + manifest.json; EXECUTE TASK / RESUME TASK (approval-gated)
DQ Data Quality Sentinel	Correctness of the numbers in Bronze, Silver and Gold	Row-count reconciliation queries across layers; FCT_BARS.is_suspect; ONNX anomaly model score; Alpaca corporate-actions API; sql/08_validation queries
XFORM Transformation Copilot	dbt models, schema changes, tests, docs	Git repo (read + open PR only); dbt parse/compile/test against a dev schema; INFORMATION_SCHEMA.COLUMNS diff
ANALYST Analyst Copilot	Business questions from humans (this is the Cortex Agent the repo already defines)	Cortex Analyst over MARKET_SEMANTIC_VIEW; Cortex Search over NEWS_SEARCH_SERVICE; freshness lookup on Gold
CURATOR Semantic Curator	Semantic model quality: metrics, synonyms, verified queries	semantic_model.yaml + semantic_view.sql (read + PR); AGENT_QUERY_AUDIT_LOG; thumbs-down feedback table
GUARD Governance and Cost Guard	Access, licensing, policy drift, credit spend, prompt injection	POLICY_REFERENCES; GRANTS_TO_ROLES; ACCESS_HISTORY; WAREHOUSE_METERING_HISTORY; CORTEX usage views; resource monitors
SCALE Capacity and Resilience	Load spikes, scaling, outages, failover and catch-up after recovery	WAREHOUSE_LOAD_HISTORY; QUERY_HISTORY queued/blocked time; Kafka broker disk and throughput metrics; connector task counts; replication/failover group status; ALTER WAREHOUSE within fixed size limits (pre-approved)
TRIAGE Incident Triage (orchestrator)	Any alert; decides which specialist agent handles it	Calls the other agents as tools; writes the incident timeline; pages a human with a summary

The problems (95), stage by stage

Each row gives the problem as it shows up, who feels it, how the assigned agent handles it, and the guardrail or human step that keeps the agent safe.

A. Market-data source (Alpaca websocket + REST)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
A1	Silent stall. Websocket stays connected but stops sending messages, so the pipeline looks healthy while prices freeze.	DE	WATCH compares last-message age against the Alpaca market clock: no messages for 60 s while the market is open is an incident. Restarts the producer through its runbook.	Auto-restart allowed once; a second stall pages a human.
A2	“No data” vs “broken”. Weekends, holidays, half-days and pre-market look identical to an outage. Alerts fire every Saturday.	DE	WATCH checks /v2/calendar before raising anything. Suppresses alerts outside trading sessions and says so in the alert text.	Calendar cached daily; a mismatch with exchange calendar is flagged.
A3	Reconnect duplicates and gaps. After a reconnect some trades repeat and some minutes are missing.	DE	DQ finds missing minute bars per symbol and duplicate trade_ids, then asks WATCH to backfill the exact window via REST.	Backfill limited to the gap window; never rewrites outside it.
A4	Rate limits (HTTP 429) during backfill. Large history pulls fail halfway.	DE	WATCH reads the failure, re-plans the backfill into smaller symbol/date chunks, resumes from the last successful chunk.	Concurrency cap set in config, not chosen by the agent.
A5	Corporate actions. A 4:1 split makes a stock look like it fell 75%. Ticker renames (FB to META) split history in two.	DE + Analyst	DQ joins suspect moves against the corporate-actions API. Labels split-driven moves as explained, proposes an adjustment-factor table.	Price adjustment is a reviewed PR, never an in-place update.
A6	Trade corrections and cancels. Exchanges cancel or correct prints; condition codes mark odd-lot or out-of-sequence trades.	DE + Analyst	XFORM proposes a condition-code filter in stg_trades; DQ reports how many trades each rule removes.	Filter rules documented and tested in dbt.
A7	Feed coverage mismatch. Free Alpaca IEX feed is a small share of total volume. Analysts compare to Bloomberg and think the data is wrong.	Analyst	ANALYST states the feed source in every volume answer and explains the gap when a user disputes a number.	Feed name stored as a column, not assumed.
A8	Vendor schema drift. A new field appears or a type changes, and validation starts rejecting every message.	DE	WATCH sees the DLQ rate spike, clusters DLQ errors by field, and hands a summary to XFORM to draft a schema update.	Schema change goes through PR + CI; DLQ is replayed after merge.
A9	Timezone and DST bugs. ET vs UTC confusion shifts the trading day; the DST weekend breaks “today”.	DE + Analyst	DQ asserts that every session has 390 minute bars between 13:30/14:30 UTC and 20:00/21:00 UTC depending on DST.	Assertion lives in dbt tests, not only in the agent.

B. Kafka and Kafka Connect

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
B1	Consumer lag growing. Connector falls behind; “real-time” data becomes minutes old.	DE	WATCH tracks lag trend, not a single value. Diagnoses cause (connector down, hot partition, slow Snowflake) and recommends a fix.	Scaling tasks or partitions needs approval.
B2	DLQ grows silently. Bad messages are routed correctly, then nobody ever looks at them.	DE	WATCH summarises the DLQ daily: top error types, affected symbols, whether the rows are replayable.	Replay of DLQ only after the root cause is fixed.
B3	Hot partitions. TSLA/NVDA traffic lands on one partition and lags while others idle.	DE	WATCH reports per-partition lag and proposes a different keying strategy.	Repartitioning is a planned change, never automatic.
B4	Retention expires before catch-up. A long outage means Kafka deletes data that never reached Snowflake.	DE	WATCH projects time-to-retention-expiry from current lag and escalates early; TRIAGE triggers REST backfill for the lost window.	Escalation threshold set as a fixed fraction of retention.
B5	Connector task failures. A task dies with a stack trace; restarts loop.	DE	WATCH reads the Connect REST trace, matches it to known causes (auth, channel invalidated, schema), applies the runbook step.	One automated restart; repeat failure goes to a human.

C. Loading into Snowflake (Snowpipe Streaming, Snowpipe, GCS)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
C1	Duplicates after a connector restart. Offset tokens reset or channels are reopened, so rows load twice.	DE	DQ checks Bronze duplicate rate per channel after every restart event and reports it; Silver MERGE dedupes by key.	Silver keeps the dedupe; the agent only reports.
C2	Batch loads fail silently. GCS file lands, COPY rejects rows, nobody notices. Or the Pub/Sub notification integration is misconfigured and nothing loads.	DE	WATCH compares files in the bucket with COPY_HISTORY and SYSTEM\$PIPE_STATUS; reports files never loaded or partially loaded.	Reload uses a named file list, never a full re-scan.
C3	Load-history dedupe. Re-uploading a fixed file with the same name is skipped by Snowpipe.	DE	WATCH recognises the “already loaded” case and proposes the correct reload path.	FORCE reloads require approval (risk of duplicates).
C4	Backfill overlaps streaming. The boundary minute exists in both paths.	DE	DQ checks the overlap window after each backfill and confirms the MERGE kept one row per key.	Overlap window defined explicitly per backfill run.
C5	Small-file cost. Thousands of tiny files raise Snowpipe overhead.	DE	GUARD reports cost per loaded GB and suggests batching thresholds.	Recommendation only.

D. Bronze to Silver (Streams + Tasks, Dynamic Tables)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
D1	Stale stream. A stream not consumed within the retention window becomes stale; the only fix is recreate, and changes in between are lost.	DE	DOCTOR watches stream staleness dates and warns days ahead; if stale, runs the recreate + gap-backfill runbook.	Recreate needs approval; the gap backfill is logged.
D2	Suspended or failing tasks. A task fails repeatedly and auto-suspends; Silver stops updating.	DE	DOCTOR reads TASK_HISTORY errors, classifies (SQL error, warehouse, permissions), fixes known causes, resumes.	RESUME TASK allowed only for classified known causes.
D3	Non-deterministic MERGE. Duplicate keys in the source batch make MERGE fail.	DE	DOCTOR identifies the duplicate keys, explains which path produced them (see C1/C4).	Fix goes into the QUALIFY dedupe, via PR.
D4	Silent nulls from TRY_TO_casts. Bad values become NULL instead of failing; data quietly disappears from metrics.	DE + Analyst	DQ tracks the NULL rate per cast column per hour and alerts on jumps.	Threshold reviewed by a human once, then fixed in config.
D5	Dynamic Table lag misses. Refreshes fall behind TARGET_LAG or fail.	DE	DOCTOR reads DYNAMIC_TABLE_REFRESH_HISTORY and explains why (upstream, warehouse size, query change).	Warehouse resizing needs approval.

E. Silver to Gold (dbt)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
E1	Full rebuilds get slow and expensive. Table materialisation rebuilds all history every run.	DE	XFORM drafts incremental versions with a late-arrival lookback window and compares outputs against the full rebuild.	Merge only if row-by-row diff is zero on a test window.
E2	2 a.m. test failure. dbt test fails; the on-call engineer has to work out which test, which rows, and whether it is real.	DE	DOCTOR reads run_results.json, runs the failing test's SQL, returns failing rows and the likely cause.	The agent never disables or weakens a test.
E3	Upstream schema change breaks models.	DE	XFORM diffs INFORMATION_SCHEMA before/after, finds affected models via the dbt DAG, opens a fix PR.	CI must pass; human review required.
E4	Grants and policies lost on rebuild. CREATE OR REPLACE drops row access policies and table grants (found and fixed in this repo in commit a480e77).	DE	GUARD runs the POLICY_REFERENCES check after every dbt run; missing policy is a sev-1.	Sev-1 pauses agent answers until fixed.
E5	Window metrics wrong at boundaries. Rolling volatility runs across the overnight gap or across a split.	Analyst	DQ tests that rolling windows reset per session; XFORM proposes partitioning by trading date.	Metric definition change requires CURATOR update too.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
E6	Docs drift. README run order no longer matches the code (also found in this repo).	DE	XFORM checks documented commands and object names against the code on every PR.	Flags only; a human edits the docs.

F. Semantic layer, Cortex Analyst, Cortex Agent (analyst-facing)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
F1	Ambiguous questions. “What's AAPL's return today?” before the open, or “return” meaning close-to-close vs open-to-close.	Analyst	ANALYST asks one clarifying question or states the assumption it used (definition, timezone, session).	Definitions come from the semantic model, not the model's guess.
F2	Wrong SQL that looks right. Bad join, wrong grain, wrong filter: a confident wrong number.	Analyst	ANALYST shows the SQL it ran; CURATOR regression-tests every verified query nightly against sql/08_validation twins.	Answers without a verified pattern are labelled as such.
F3	Two sources of truth. semantic_model.yaml and semantic_view.sql define metrics separately and drift apart.	DE + Analyst	CURATOR diffs the two definitions on every PR and fails CI on mismatch.	Single owner signs off on metric changes.
F4	Routing errors. Agent sends a “why did it move” question to SQL only, or a numeric question to search.	Analyst	CURATOR keeps a labelled set of routing examples and scores the agent on it after each instruction change.	Routing accuracy target agreed before rollout.
F5	RAG problems. News index lags (TARGET_LAG 1 h), wrong-ticker articles (“F” is Ford; “Apple” in a recipe), missing citations.	Analyst	ANALYST filters search by symbol attribute, always cites, and states article timestamps. DQ measures retrieval precision on a labelled set.	No citation means no news claim in the answer.
F6	“The data is wrong” that is actually the 15-minute delay. Delayed-entitlement users see older prices than a live screen.	Analyst	ANALYST states the as-of timestamp and entitlement in every price answer.	Policy enforced in Snowflake; the agent only explains it.
F7	Suspect bars in aggregates. One bad print distorts daily return or volatility.	Analyst	ANALYST checks is_suspect rows in scope and mentions them; offers the figure with and without them.	Suspect rows are flagged, never silently dropped.
F8	Waiting on engineering for every new metric.	Analyst	CURATOR mines unanswered questions from the audit log and drafts new metrics + verified queries as a PR.	Data engineer reviews every new metric.
F9	Slow or expensive answers. Agent calls time out or cost too much per question.	Analyst + DE	GUARD tracks latency and credits per question type; CURATOR adds pre-aggregated views for the heaviest patterns.	Per-role statement timeout already in sql/09_guardrails.

G. Governance, security and cost

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
G1	Licence leakage. A role that shouldn't see real-time prices gets them through a new grant or a new table.	DE	GUARD diffs grants daily and checks every new Gold table for the row access policy.	New grants to REALTIME_DESK always need human approval.
G2	Prompt injection through news. A news article contains text like “ignore your instructions and...”, and the agent reads it as a command.	DE	GUARD screens retrieved text and the agent treats search results as data only; the agent's role has read-only grants, so even a successful injection cannot change data.	Least-privilege role is the real defence, not the screen.
G3	Runaway spend. A loop of agent queries or a bad dbt model burns credits overnight.	DE	GUARD watches metering per warehouse and per role and suspends at the resource-monitor threshold.	Resource monitor in sql/09_guardrails is the hard stop.
G4	Audit gaps. Can't prove who asked what and what data they saw.	DE	GUARD reconciles AGENT_QUERY_AUDIT_LOG with QUERY_HISTORY and reports missing entries.	Audit table is append-only.
G5	Secret rotation. Alpaca keys and Snowflake key pairs expire or leak.	DE	GUARD tracks key age and reminds before expiry.	Agents never read or store secret values.

I. Data volume and overload

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
I1	Market-open and close spikes. Volume at 9:30 and 16:00 ET is many times the midday rate; producer, Kafka and Snowpipe Streaming all lag at once.	DE	SCALE pre-scales connector tasks and warehouses before the open using the market calendar, then scales back after the spike.	Schedule pre-approved; hard size cap.
I2	Event-day surges. CPI, FOMC, earnings or a meme-stock day brings sudden, unplanned volume far above normal.	DE	SCALE compares throughput with the same time-of-day baseline, scales within limits, and warns DQ to expect late data.	Resource-monitor credit ceiling still applies.
I3	Producer backpressure. The producer's send buffer fills; it either drops messages or runs out of memory.	DE	WATCH tracks buffer use and send errors and alerts before the buffer is full.	Producer configured to block, never silently drop.
I4	Kafka broker disk full. A surge plus long retention fills disks; brokers stop accepting writes.	DE	SCALE projects time-to-full from the fill rate and proposes retention or disk changes early.	Retention/disk changes need approval.
I5	Snowpipe Streaming throttling. Client hits throughput or channel limits and starts returning errors.	DE	WATCH reads client errors; SCALE proposes more channels or connector tasks.	Changes go through config review.
I6	Warehouse queuing. Tasks, dbt and agent queries compete for compute; queries queue and Silver falls behind at peak.	DE	SCALE reads queued time in QUERY_HISTORY and proposes separate or multi-cluster warehouses (ANALYST_WH is already separate).	Bigger warehouses need approval.
I7	Many analysts at once. Morning question bursts hit Cortex and the warehouse together; answers time out.	Analyst	GUARD rate-limits per user; SCALE enables multi-cluster scaling for ANALYST_WH within limits.	Per-role statement timeout stays on.
I8	Huge scans. A broad question makes the agent scan years of trades.	Analyst + DE	ANALYST adds a default date range and says so; SCALE suggests clustering keys for the common filters.	Statement timeout is the hard stop.
I9	Storage growth from churn. MERGE every minute multiplies Time Travel and Fail-safe storage.	DE	GUARD reports storage per table including Time Travel bytes and proposes transient Bronze or shorter retention.	Retention changes need approval.

J. Outages and recovery (server down)

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
J1	Alpaca outage or degraded feed. Vendor is down or sending partial data during market hours.	DE + Analyst	WATCH detects it, records the window in an OPS outage table; ANALYST tells users that window is incomplete; backfill runs when the vendor recovers.	Outage windows are recorded, never hidden.
J2	Kafka broker down or quorum lost. Producer can't write.	DE	WATCH detects write failures; TRIAGE pages with the broker state and the data at risk.	Agents never restart cluster nodes on their own.
J3	Kafka Connect worker crash or rebalance storm. Tasks keep moving between workers and nothing loads.	DE	WATCH spots repeated rebalances in the Connect logs and recommends the runbook fix.	One automated restart, then a human.
J4	Snowflake unavailable (region incident, or a warehouse can't resume).	DE	TRIAGE confirms with a probe query and the status page, and works out how long Kafka retention can buffer (see B4). Failover to a second region only if replication is set up.	Failover is always a human decision.
J5	Cortex unavailable or model not available in the region. Agent questions fail.	Analyst	ANALYST falls back to verified-query results or the direct Cortex Analyst path and says it is in degraded mode.	No made-up answers in degraded mode.
J6	GCS or Pub/Sub notification outage. Files land but no notification arrives, so nothing loads.	DE	WATCH compares the bucket listing with COPY_HISTORY and proposes ALTER PIPE ... REFRESH for the missed window.	Refresh scoped to the window; approval needed.
J7	Recovery storm. After an outage everything catches up at once and causes a second overload.	DE	SCALE paces the catch-up; DQ confirms completeness for the outage window afterwards.	Catch-up plan logged with the incident.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
J8	Partial outage. Only some symbols or one partition stop; the global “last message” metric still looks fine.	DE	DQ checks freshness per symbol, not just overall.	Per-symbol thresholds account for illiquid names.
J9	Bad deploy takes the pipeline down. A SQL or dbt change breaks tasks or corrupts a table.	DE	DOCTOR links the failure to the change (git history + QUERY_HISTORY) and proposes a rollback or a Time Travel restore (AT/BEFORE, UNDROP).	Restores need approval.

K. Time and timestamp matching

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
K1	Event time vs load time. trade_ts (when it happened) and _loaded_at (when it arrived) get mixed up; metrics use the wrong one.	DE + Analyst	DQ tracks the lateness distribution (load minus event time); XFORM checks every model aggregates on event time.	Event-time rule written in AGENTS.md.
K2	Producer clock skew. A host without NTP stamps wrong times; latency comes out negative or impossible.	DE	WATCH flags negative or implausible latencies and names the host.	Fix is NTP on the host, not adjusting data.
K3	Bar timestamp convention. Alpaca stamps a minute bar at the start of the minute; other sources use the end. Joins are off by one minute.	DE + Analyst	DQ asserts the convention on each source; CURATOR documents it in the semantic model.	Convention converted at Silver, once.
K4	Trades don't match bars. Bars rebuilt from trades differ from vendor bars (feed coverage, condition filters, boundary inclusive/exclusive).	DE + Analyst	DQ reconciles rebuilt vs vendor bars within a tolerance and explains each difference type.	Tolerances agreed with analysts.
K5	Late data after aggregates are built. A bar is final, then a correction arrives.	DE	DQ measures how late data really arrives so the incremental lookback window (E1) is sized from evidence.	Lookback set in config, reviewed.
K6	Out-of-order arrival. Messages from different partitions arrive out of sequence; logic that assumes arrival order breaks.	DE	XFORM checks every window function orders by event time with a tie-breaker (trade_id).	Enforced by a dbt test.
K7	Precision truncation. Nanosecond timestamps cut to milliseconds or seconds create duplicate (symbol, ts) keys and MERGE collisions.	DE	DQ checks key uniqueness and timestamp precision per column.	Columns declared TIMESTAMP_NTZ(9).
K8	Session timezone vs NTZ. CURRENT_TIMESTAMP() is TIMESTAMP_LTZ; comparing it with a UTC NTZ column depends on the session TIMEZONE. With Snowflake's default America/Los_Angeles, a 15-minute delay rule can hide hours of data. <i>This repo's DELAYED_DATA_POLICY uses that comparison.</i>	DE + Analyst	GUARD tests the policy under several session timezones and fails if the visible window changes.	Fix: compare against a UTC NTZ value (e.g. SYSDATE()); verify on live Snowflake.
K9	News-to-price alignment. Using updated_at instead of published_at, or the wrong window, pins the wrong article to a price move.	Analyst	ANALYST searches only news published shortly before the move and states the time gap.	No causal claim without a timestamp.
K10	Trading-day boundary. date_key is the UTC day, so after-hours bars past 8 p.m. ET fall on the next day.	Analyst	DQ flags bars whose UTC date differs from the exchange date; XFORM proposes an exchange-time date_key.	Metric change also updates CURATOR.

L. Market structure and data correctness

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
L1	Trading halts and LULD pauses. No trades for minutes looks exactly like an outage.	DE + Analyst	DQ checks halt news before raising a gap; WATCH suppresses the outage alert for that symbol.	Halt windows stored with source.

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
L2	Listings, delistings, symbol changes. dim_symbols goes stale; the agent says “no data” for a valid ticker.	DE + Analyst	DQ diffs the symbol master daily and reports adds, removals and renames.	History kept for inactive symbols.
L3	Bad prints the 10% rule misses. Outliers within 10%, or legitimate moves above it.	DE + Analyst	DQ scores bars with the ONNX anomaly model alongside is_suspect and reports disagreements.	Flag only; never delete.
L4	Illiquid gaps. Thin symbols have minutes with no trades; teams disagree on forward-fill.	Analyst	CURATOR documents one gap rule; XFORM implements it consistently.	Rule visible in metric descriptions.
L5	Extended hours mixed with the regular session. Pre- and after-hours distort volume and returns.	Analyst	ANALYST defaults to the regular session and says so; session is a semantic-model dimension.	Default stated in every answer.
L6	Unit mismatch. A new source reports cents or lots instead of dollars or shares.	DE	DQ range-checks price and size per symbol against history.	New sources start in quarantine.

M. Concurrency, reprocessing and change

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
M1	Task overlap. A MERGE takes longer than its schedule; runs are skipped and lag builds.	DE	DOCTOR compares task duration with the schedule and recommends a new interval or warehouse size.	ALLOW_OVERLAPPING_EXECUTION stays false.
M2	Lock contention. A backfill MERGE and the streaming MERGE hit the same table; lock waits and timeouts follow.	DE	DOCTOR finds blocked queries and schedules backfills outside streaming peaks.	Backfill windows planned in advance.
M3	Reprocessing history. A bug fix means rebuilding months of data without double counting.	DE	DOCTOR plans the rebuild in a zero-copy clone, compares results, then proposes a swap.	Swap needs approval; old table kept via Time Travel.
M4	Real-time vs batch disagree. The streaming path and the backfill give different numbers for the same day.	DE + Analyst	DQ reconciles them daily; after T+1 the batch path is the source of truth.	Rule documented for analysts.
M5	Environment drift. Dev and prod objects differ, so a change works in dev and fails in prod.	DE	XFORM diffs GET_DDL output between environments before deploys.	Deploys blocked on drift.
M6	Bad data reaches Gold. Wrong numbers are already published.	DE + Analyst	DOCTOR proposes a Time Travel restore; ANALYST flags answers given during the bad window (from the audit log).	Restore needs approval; users told.

N. Analyst trust and reproducibility

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
N1	Same question, different answer. Asked at 10:05 and 10:30, numbers differ because of late data and the delay window.	Analyst	ANALYST gives an as-of time with every answer; the audit log keeps the SQL so it can be re-run with Time Travel AT that time.	Audit log is append-only.
N2	Look-ahead bias. Backtests use corrected prints or news that weren't available at the time.	Analyst	CURATOR provides point-in-time views filtered on _loaded_at.	Backtest questions use point-in-time views.
N3	Survivorship bias. Delisted symbols missing from history inflate returns.	Analyst	ANALYST includes inactive symbols for historical questions and warns when the universe changed.	dim_symbols keeps inactive rows.
N4	Metric disputes. Teams calculate VWAP or return differently.	Analyst	CURATOR keeps one definition per metric, with synonyms that map to it.	Metric owner signs off.
N5	Dashboard load. BI tools refresh Gold every few seconds.	DE	SCALE routes BI to its own warehouse and relies on the result cache; GUARD reports top consumers.	Recommendation only.
N6	Questions outside the data. Fundamentals, options, other markets: the risk is a made-up answer.	Analyst	ANALYST says the data isn't available; CURATOR logs it as demand for new data.	No answer without a source.

O. Operations and people

#	Problem and how it shows up	Who	How the agent solves it	Guardrail
O1	Root cause spans six systems. A stale Gold number could start in Alpaca, Kafka, Snowpipe, a task, dbt or the agent.	DE	TRIAGE walks the chain upstream, asking each specialist agent for its health, and returns the first broken link with evidence.	Summary goes to a human; actions stay with specialists.
O2	Reconciliation across layers. Kafka offsets vs Bronze rows vs Silver vs Gold don't add up.	DE	DQ runs per-hour count reconciliation and reports the layer where rows disappear.	Tolerances set per layer (dedupe expected in Silver).
O3	Alert fatigue. Dozens of alerts for one root cause.	DE	TRIAGE groups related alerts into one incident with a timeline.	Raw alerts still logged.
O4	Tribal knowledge and missing runbooks. Only one person knows how to fix a stale stream.	DE	Every agent fix is written as a runbook step first; TRIAGE writes a post-incident note proposing a new runbook entry.	Runbooks reviewed and versioned in git.
O5	No CI, no tests (this repo's current gap). Changes reach Snowflake untested.	DE	XFORM runs dbt parse/test and Python tests on every PR once CI exists.	Build pytest + CI first (README Target items).

How each agent is trained and tested

Scenarios reuse what the repo already has: `generate_sample_data.py` and `generate_sample_news.py` produce clean data, a small fault-injection layer corrupts it in known ways, and `replay_sample_data.py` plays it through Kafka. Every scenario has an expected diagnosis, so it can be scored.

Agent	Knowledge it is given	Training scenarios	Pass criteria (evals)
WATCH	Runbooks for each stage of ingestion; market calendar rules; Kafka/Connect error catalogue.	Fault injection with <code>replay_sample_data.py</code> : kill producer, drop minutes, duplicate bursts, malformed rows, pause connector.	Detects fault within N minutes; correct root cause; correct runbook step; zero alerts on closed-market periods.
DOCTOR	Task/stream/dbt failure catalogue with fixes; dbt DAG.	Break things on a dev account: let a stream go stale, suspend a task, insert duplicate keys, fail a dbt test.	Correct classification; failing rows returned; never weakens a test.
DQ	Expected shapes: 390 bars per session, price bands, null-rate baselines, corporate-action rules.	Inject splits, bad prints, gaps, halts, timezone shifts, clock skew, ms truncation, late and out-of-order rows into generated sample data.	Precision/recall on injected faults; explained vs unexplained moves.
XFORM	AGENTS.md conventions; dbt style; schema contracts.	Historical schema changes replayed as tasks; known bugs (e.g. GOLD_GOLD, lost policy) as test cases.	PR passes CI; reviewer accepts without major changes.
ANALYST	Semantic model, metric definitions, delay/entitlement rules, citation rules.	Question bank: verified queries, ambiguous questions, adversarial questions, news questions.	Numeric answers match sql/08 twins; routing accuracy; citation present; correct clarification rate.
CURATOR	Both semantic definitions; audit log; feedback table.	Seed failed/unanswered questions; seed YAML vs semantic-view mismatches.	Mismatch detection rate; accepted metric PRs.
GUARD	Policy model (who sees real-time); cost budgets; injection patterns.	Add a grant, create an unprotected table, plant injection text in <code>sample_news.csv</code> , run a cost spike, query the policy under different session timezones.	Every planted violation detected; no false sev-1.
SCALE	Time-of-day volume baselines; macro-event calendar; capacity limits of each component.	<code>replay_sample_data.py</code> at 10x-50x --speedup; stop the Kafka container; suspend a warehouse; fill a test broker's disk.	Lag recovers within the agreed time; no credit overrun; no data lost; catch-up paced.
TRIAGE	Dependency graph of the stack; escalation policy.	Composite incidents built from the scenarios above.	First broken link identified; one incident per root cause.

Feedback loop in production

- Every agent action and answer is logged (AGENT_QUERY_AUDIT_LOG for ANALYST; an incident log for the others).
- Humans mark answers or actions as right or wrong; wrong ones become new eval cases with the corrected answer.
- The eval suite runs on every change to an agent's instructions, tools or the semantic model. A regression blocks the change.
- Agents move up an autonomy level only after a sustained pass rate on their evals and in production review.

Building the agents on Snowflake

The agents don't need a separate platform. Each piece maps to a Snowflake feature, so detection, action, approval, logging and evaluation all sit next to the data they protect.

Piece	Snowflake feature	Why
Triggers	Snowflake ALERT objects (CREATE ALERT ... IF (EXISTS (query)) THEN ...) on a schedule; Data Metric Functions attached to Bronze, Silver and Gold tables	Detects the condition inside Snowflake and wakes the right agent. No external scheduler is needed for in-Snowflake problems.
Agent brain	Cortex Agents (the same mechanism as MARKET_DATA_AGENT), or an external agent calling Snowflake through the repo's MCP server	ANALYST and CURATOR fit Cortex Agents naturally. WATCH has to reach Kafka and Alpaca, so it runs outside and writes its findings into Snowflake.
Tools	Stored procedures, one per runbook step (for example RESUME_TASK_SAFE, RECREATE_STREAM_WITH_BACKFILL), owned by a dedicated OPS role	The agent can only do what a procedure allows; the procedure checks preconditions and logs every call.
Approval	An OPS.PENDING_ACTIONS table plus a Streamlit in Snowflake page where a human approves or rejects	Implements L2 autonomy: the agent proposes, a human approves, the procedure runs.
Observability	Event table (Snowflake Trail) for procedure and agent logs; ACCOUNT_USAGE and INFORMATION_SCHEMA views	One place to reconstruct every incident and every agent decision.
Notification	Notification integration (email, or a webhook to Slack)	Alerts and approval requests reach humans without another system.
Evals	OPS.EVAL_CASES and OPS.EVAL_RESULTS tables; scenarios replayed into a zero-copy CLONE of the database	Fault scenarios run against a clone, so training runs never touch production.
Security	A separate least-privilege role per agent (OPS_WATCH, OPS_DOCTOR, ...); ANALYST_AGENT stays read-only	A wrong or compromised agent can only do what its role allows.

One problem end to end in Snowflake: a stale stream (D1)

- **Detect.** A scheduled ALERT checks SHOW STREAMS output for any stream whose stale_after date is less than 2 days away.
- **Diagnose.** The alert calls DOCTOR, which reads TASK_HISTORY and finds the consuming task has been suspended since a failure 9 days ago.
- **Propose.** DOCTOR writes a row to OPS.PENDING_ACTIONS: fix the task error, resume it, and consume the stream before it goes stale. Evidence attached.
- **Approve.** The on-call engineer approves it on the Streamlit in Snowflake page.
- **Act.** The RESUME_TASK_SAFE procedure checks preconditions, resumes the task, and logs to the event table.
- **Verify.** DQ confirms Silver row counts have caught up with Bronze. The incident closes, and the case is added to OPS.EVAL_CASES so the next DOCTOR version is tested on it.

Rollout order

Built in the order that matches the repo's README plan, so each step has real infrastructure to test against:

- **Step 0: prerequisites.** Connect the live Snowflake trial, local Kafka, GCS bucket and Alpaca key; run every SQL file for real; build the pytest suite and CI (README “Target” items). Agents can't be evaluated against a pipeline that has never run.
- **Step 1: fault-injection harness.** Extend the sample generators so each problem in this document can be triggered on demand.
- **Step 2: L0/L1 observers.** WATCH, DQ, GUARD and SCALE (observe only) first, read-only. They catch the problems that silently damage data.
- **Step 3: analyst side.** ANALYST evals against the sql/08 twins, then CURATOR for the semantic model.
- **Step 4: repair agents.** DOCTOR, XFORM and SCALE actions (scaling, catch-up), approval-gated (L2).
- **Step 5: TRIAGE** once the specialists are reliable enough to call as tools.